Experiment - 1

Aim Write a Python program to understand SHA and Cryptography in Blockchain

Tasks Performed

1. **Hash Generation using SHA-256:**
   Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

2. **Target Hash Generation with Nonce:**
   Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

3. **Proof-of-Work Puzzle Solving:**
   Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. **Merkle Tree Construction:**
   Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program 1: Hash Generation Using hashlib Library

```
import hashlib
def create_hash(string):
    hash_object = hashlib.sha256()
    hash_object.update(string.encode('utf-8'))
    hash_string = hash_object.hexdigest()
    return hash_string
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

OUTPUT :

```
Enter a string: R
Hash: 8c2574892063f995fdf756bce07f46c1a5193e54cd52837ed91e32008ccf41ac
```

Program 2: Generating Target Hash with Input String and Nonce

```
import hashlib
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
hash_string = input_string + nonce
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()
print("Hash Code:", hash_code)
```

OUTPUT:

```
Enter a string: O
Enter the nonce: 4
Hash Code: cdf5313889b8d01277d5e0eef37ad9ecd5446e386ef792dbcefff9674d58f458
```

Program 3: Solving Cryptographic Puzzle for Leading Zeros

```
import hashlib
def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros
    while True:
        hash_string = input_string + str(nonce)
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()
         if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce
            nonce += 1
input_string = input("Enter the input string: ")
num_zeros = int(input("Enter number of leading zeros required: "))
expected_nonce = find_nonce(input_string, num_zeros)
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

OUTPUT:

```
Enter the input string: S
Enter number of leading zeros required: 4
Hash: 0000fbaccfc71429cd46551238549ac5cf1b33a78f05a4943ec395a11fb6e0bf
Input String: S
Leading Zeros: 4
Expected Nonce: 1156
```

Conclusion:

This program demonstrates how Proof-of-Work works in blockchain by finding a nonce that produces a hash with required leading zeros. It shows the importance of SHA-256 hashing and computational effort in block validation. Thus, the experiment provides a basic understanding of mining and security in blockchain systems.